

Shaving the PromQL Yak



~6 PromQL functions to escape from analytical workarounds.

A yak shave that started with 'are these two timeseries related?' and ended with a need to have a matrix solver inside a query engine 🤖

JuanJo Ciarlante @jjo¹

Principal Eng @GrafanaLabs, platform-team

PromCon EU 2026

¹note to `self`: don't forget to thank the prom devs for merging [limitk](#), [limit_ratio](#), and [integral](#) support ✨

Great at “what”, hint me+ on “possible why”s

The Core Problem

Ergonomics, but "more".

Operators ask:

- "Are these signals related?"
- "Is this trend unusual?"
- "How long until we hit a threshold?"

The Four Horsemen of Workarounds

Contrived Nested Queries

But some math can't even be done in a single shot -v

Recording Rules

That can't be dependent on each other (eval time here also)

\${tool} Exports

Exporting to JSON, CSV to post-process them in R, python

Visual "diff"

Train your eye to spot the correlations!



Bring some maths to the data, "just" the QL

Pure promQL: no new storage format, sidecar, or rewrite ... just `--enable-feature=promql-experimental-functions`

time_to_threshold

How long until we cross?

From: "Are we there yet?(repeat) "
To: "How long until there?".

Ergonomics + some savings vs
`(T-predict_linear())/deriv()`

zscore / robust_zscore

Is this unusual?

How weird is this sample, in standard deviations? The robust flavour can't be bullied by the outlier.

Later found it's **regurgitation~ish** from 2023 :)

ewma_over_time

What's the signal under the noise?

A smoother with a goldfish memory: spots a change sooner than `avg_over_time`, twitches a bit more.

correlation_over_time

Do they move together?

Do two wiggles dance together? A grade from -1 to 1. No units, no promises, no causation.

regression_over_time lm_over_time

What value should I expect?

Draws the best line through y vs x. You get a slope, a prediction, and the residual: how far reality strays from the line.

timeseries_gen [the ugly]

Synthetic series generation

Conjures series *out of thin air* for testing rules and joins. Handle with oven gloves.



time it, un-smooth it, judge it

time_to_threshold()

The countdown clock.

Seconds until the line crosses your limit.
A flat line gets NaN, not a cheerful $4e+18$ seconds.

ewma_over_time()

The goldfish smoother.

Unlike **avg_over_time**, it can weight recent samples more: it reacts sooner but lets more noise through, **alpha** parameter sets the trade-off.

zscore() & robust_zscore()

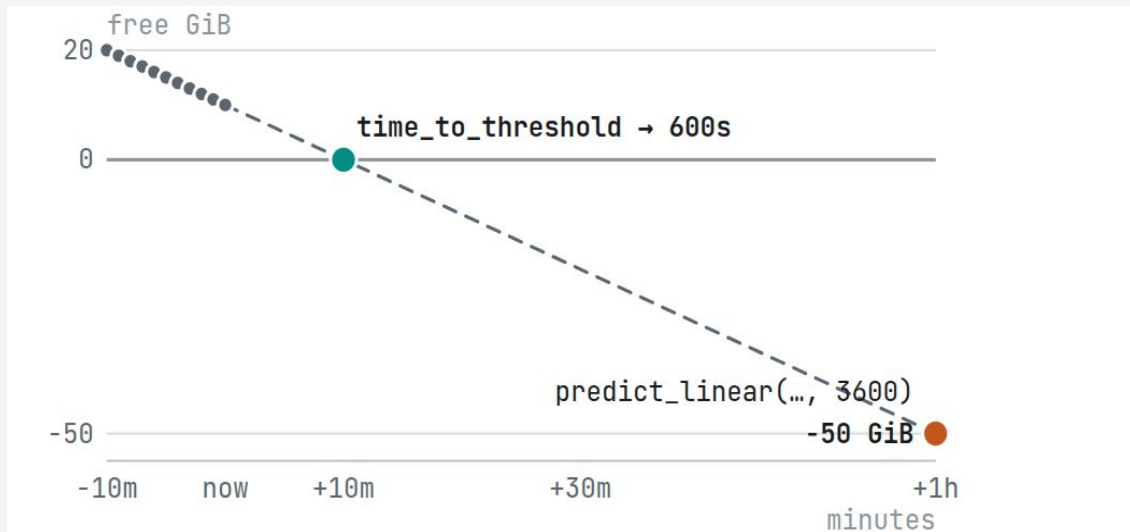
The weirdness meter, revamped
[EHLO @beorn7 ;]

Robust variants use the **median and median absolute deviation**, so spikes don't inflate the baseline and hide themselves.



How it works · `time_to_threshold()`

Same ruler, other end



predict_linear: "v at now + Δt ?"

time_to_threshold: "when does v hit T?"

actually, possible now with:

```
(T - predict_linear(x[6h], 0))  
/  
deriv(x[6h])
```

predict_linear makes you guess a horizon,
i.e. *value-wise*

How alive would I be, X time from now?

time_to_threshold answers the question you
actually had, i.e. *time-wise*

How long have I got, doctor?



Who's running out of disk?

```
persistentvolumeclaim="mysql-data-mariadb-server-0",
```

8.770184049601097

```
persistentvolumeclaim="storage-prometheus-stack-grafana-0",
```

15076888.514976032

```
sort(  
  time_to_threshold(  
    kubelet_volume_stats_available_bytes[6h],  
    0  
  ) / 86400 > 0  
)
```

Days until each volume hits zero. mariadb: 8.8 days, time to buy the DBA a coffee :)

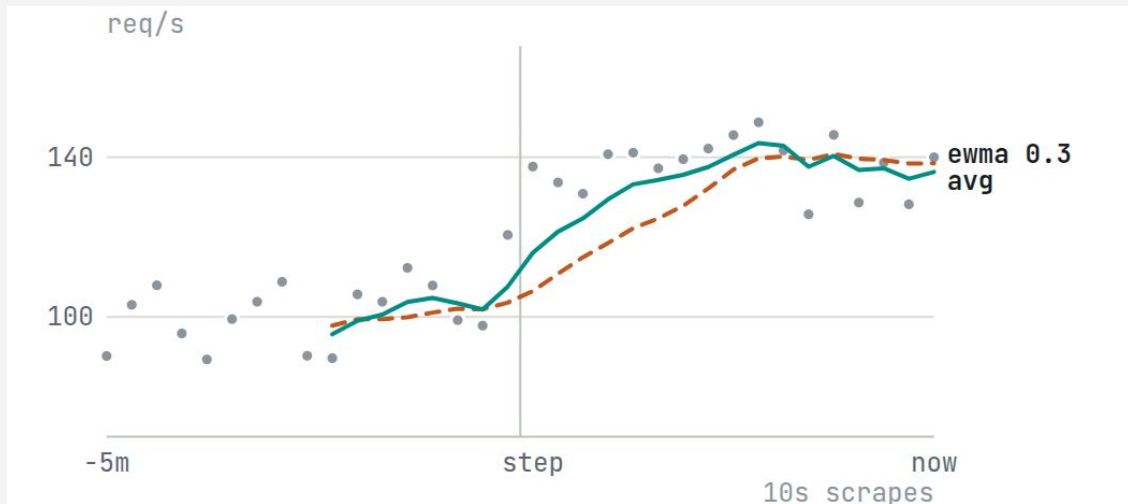
Grafana's volume: 15 million days, about 41K years, so the hardware retires first, I'd guess.

Full query: appendix A3.



How it works · ewma_over_time()

The goldfish and the elephant



$$\text{new} = \alpha \cdot \text{sample} + (1 - \alpha) \cdot \text{old}$$

every scrape, old memories
fade a little more

$\alpha = 1$ → pure goldfish (last value)
 α small → calm, slow, late

`avg_over_time` is the elephant: every
sample in the window counts the same,
until it falls out.

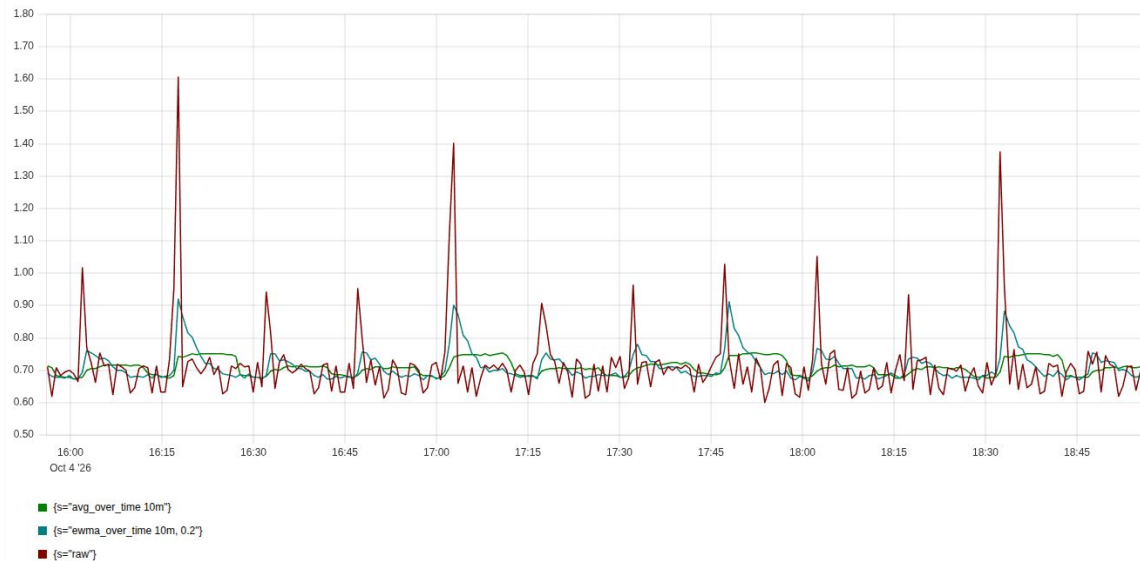
`ewma_over_time` is the goldfish: it mostly
cares about what just happened. Quicker to
notice a change, twitchier about noise.

α picks your fish.



Live on Prometheus · `ewma_over_time()`

Meet the ghost plateau



```
ewma_over_time(cpu_rate[10m:30s], 0.2)  
avg_over_time(cpu_rate[10m:30s])
```

This Prometheus' own CPU, 3h.

At every compaction spike the EWMA jumps and fades.

The plain average builds a flat 10-minute mesa that never happened, then falls off a cliff when the spike leaves the window. Full query: appendix A1.



How it works · `zscore()` & `robust_zscore()`

The outlier that grades its own exam



```
zscore      = (x - mean) / stddev  
robust_zscore = (x - median) / spread*
```

* spread = median distance to the median

with n series, **zscore can never beat** $\sqrt{n - 1}$

8 series → at most 2.65

101 series → at most 10 :P

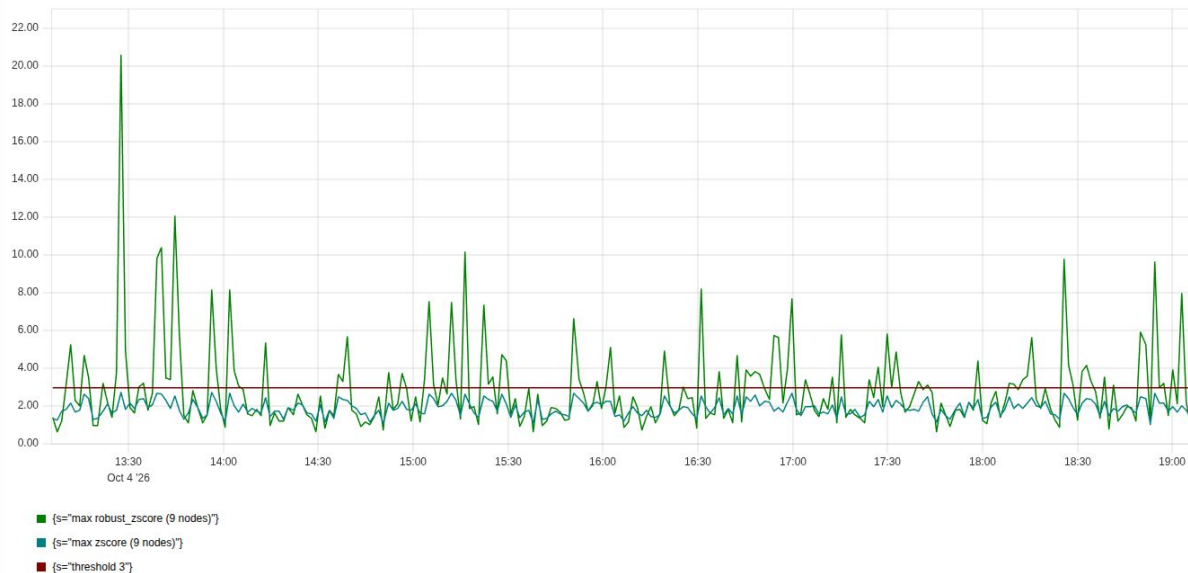
One "bad" instance **drags the mean toward itself** and **stretches the stddev**: it's grading its own exam.

The median doesn't care. Bonus trap: with 8 series, $\text{zscore} > 3$ simply cannot happen.



Live on Prometheus · `zscore()` & `robust_zscore()`

zscore > $\sqrt{n}$ is a 🦄



```
max(zscore(load1{9 nodes}))  
max(robust_zscore(load1{9 nodes}))
```

ceiling with 9 nodes:
 $\sqrt{8} \approx 2.83$

The fleet's loudest node, 6h.

Plain `zscore` bumps its head on the $\sqrt{8}$ ceiling and never touches the red line.

robust_zscore yells all the way up to 20. Same data, same node.

Full query: appendix A2.



correlation_over_time() Overview

Supported Methods

One number from -1 to 1: do these two wiggle together?
No units, no causation, no refunds.

- **pearson**: how well the best-fit (least-squares) line explains the dots cloud (for **linear** relationships)
- **spearman**: **rank-based for non-linear** rising trends
- **kendall**: how often **both series step the same way** (↗ or ↘ -vs- ↖ or ↙)

The Catch

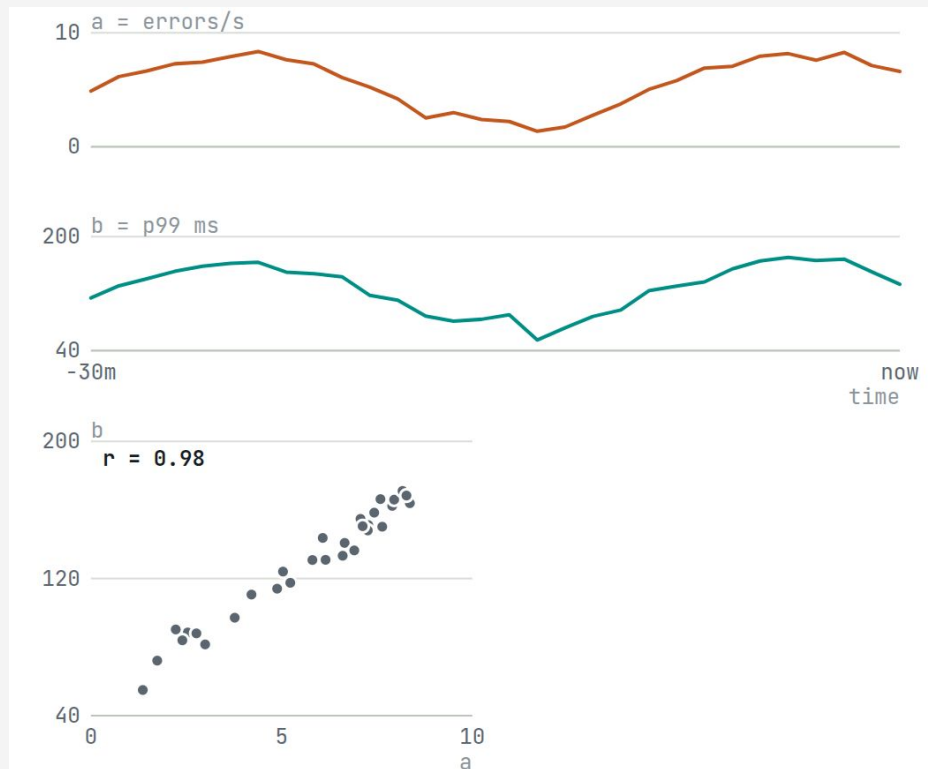
Everything going up together looks related. So do ice cream and sunburn.

- It has **no magnitude**, ie [-1, 1]
- * does **not equal causation** (before you grill me 0:)
- don't correlate apples with oranges: a snapshot gauge vs a smoothed average won't match (even when one drives the other).



How it works · `correlation_over_time()`

Two wiggles in, one number out



plot a against b,
one dot per timestamp

$r = +1$ dancing in lock-step
 $r = 0$ total strangers
 $r = -1$ mirror image

"spearman": same, on ranks
"kendall": on x-point same direction (↗)

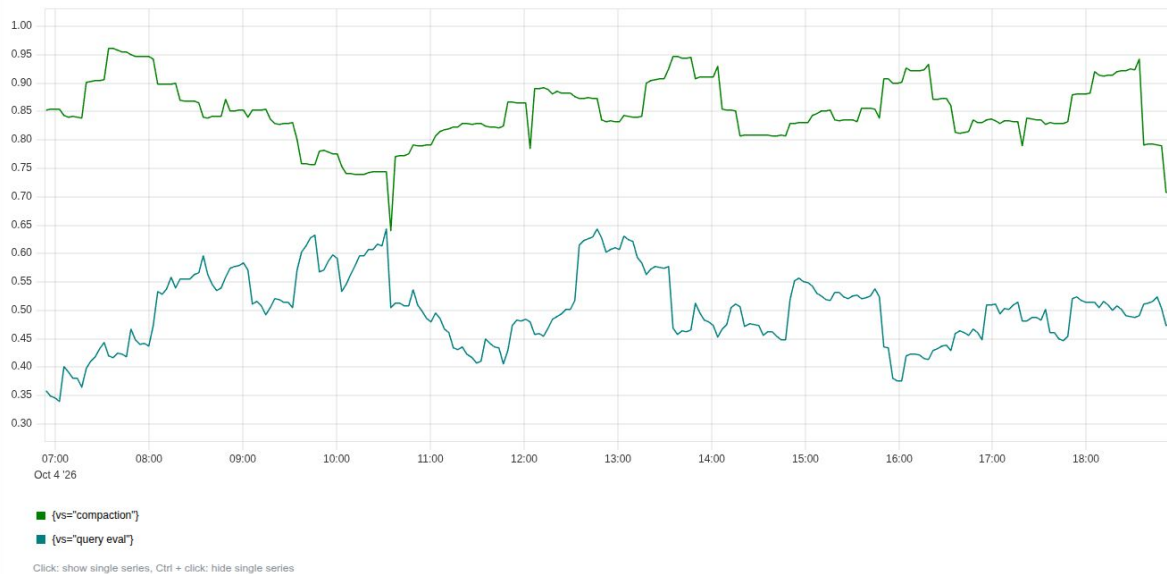
`correlation_over_time` looks at the dot cloud and grades it from -1 to 1 . No units, no causation, no promises: just *"these two move together"*.

Great for rounding up suspects, terrible as a verdict.



Live on Prometheus · `correlation_over_time()`

My prometheus CPU, is query- driven?



```
correlation_over_time(  
  cpu_rate[1h:1m],  
  compaction_rate[1h:1m]  
)
```

... and again vs `query_eval` time (rate)

Rolling 1h correlation with this Prometheus' CPU:

- Compaction: 0.75–0.95, caught red-handed
- Queries (`query_eval`, mostly rules): 0.35–0.65, innocent of the spikes.

One ~homelab at Uni, with ~23 API queries/hour, so your mileage *will* vary.

Full query: appendix A4.



From Correlation to Expected Values

Correlation vs. Regression

While correlation tells you whether signals move together, regression tells you **how much**.

regression_over_time outputs slope, intercept, prediction, or r^2 .

r

Correlation

Measures strength & direction of a linear relationship between signals.

f

Regression

Estimates how much one signal moves per unit of another, and the value to expect.

Expected Values in Action

Ask What the Load Explains

From: "Latency is 300ms. Is that bad?"

To: Read the residual: "latency is higher than traffic explains."

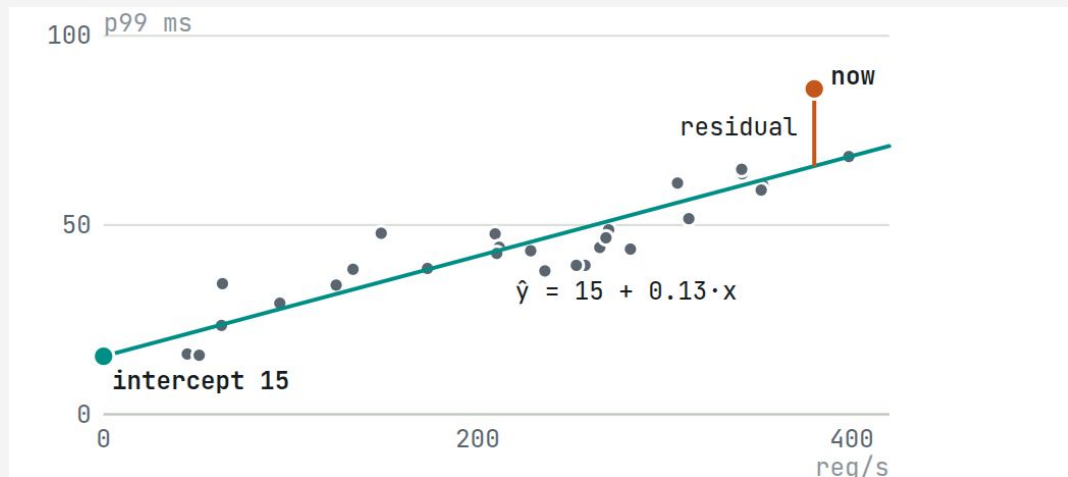
Load-Aware Baselines

- The expected value scales with traffic.
- Fitted per instance, not one global guess.
- Experimental: a lens for exploring, not a verdict.



How it works · `regression_over_time()` & `lm_over_time()`

That ``Add trendline`` – as a PromQL function



one driver:

$$y \approx a + b \cdot x$$

b = y per unit of x

a = y when x is 0

many drivers:

$$y \approx a + b_1 \cdot x_1 + b_2 \cdot x_2 + \dots$$

one weight per ingredient

`regression_over_time` "draws" the best line through y vs x and hands you the slope, the intercept, the prediction or r^2 .

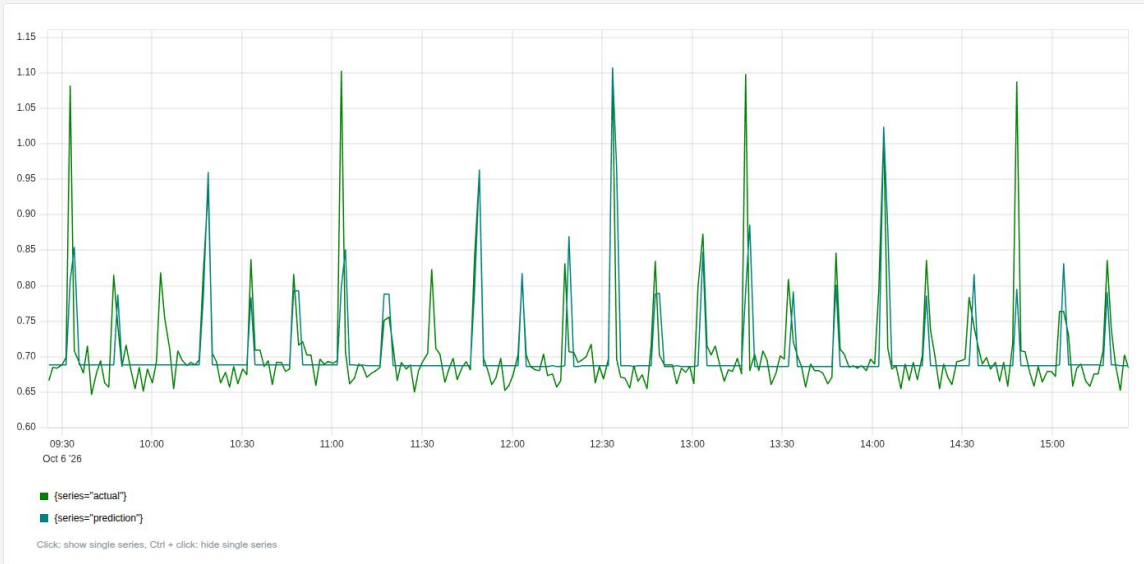
`lm_over_time` does the same with a whole recipe of drivers.

With a single driver they agree to the last digit.



Live on Prometheus · `regression_over_time()`

What does a compaction cost, cpu-wise?



```
regression_over_time(  
  cpu_rate[6h:1m],  
  compaction_rate[6h:1m],  
  "prediction"  
)
```

intercept $\approx$ 0.69 cores
slope $\approx$ 0.6 core
 $r^2 \approx$ 0.75

This Prometheus' own CPU, 6h. Intercept: the always-on base cost, about 0.7 cores. Slope: the **compaction** surcharge, about **0.6 of a core while it runs**. A plain ratio can't split that.

Spikes the line misses are someone else:
`lm_over_time` brings more suspects

Full query: appendix A5.



From one driver to many

One driver

`regression_over_time`(y, x)

$$y = a + b \cdot x$$

One driver x: a is the always-on part, b is the cost per unit of x.

On this box: CPU $\approx 0.69 + 0.6 \cdot \text{compaction_rate}$.

Many drivers

Same idea, one weight per driver:

$$y = b_0 + b_1 \cdot x_1 + b_2 \cdot x_2 + \dots$$

b_0 is still the always-on part; each of $b_1, b_2, \dots$ **coefficient** (~weight) is how much **y** moves per unit of each driver.

Written in PromQL

```
lm_over_time("lm",  
  y,  
  (x1 or x2 or ...),  
  "d"  
)
```

Each value of label d is one driver:
 $x_1, x_2, \dots$

Output: one series per driver ($b_1, b_2, \dots$), plus (intercept) = b_0 and (r^2).

"d" is short for "driver"



Two suspects, two different crimes

<code>{d="compaction"}</code>	0.620205454123473
<code>{d="query_eval"}</code>	0.3435084010857773
<code>{d="(intercept)"}</code>	0.5958541903380914
<code>{d="(r2)"}</code>	0.7542995502022491

```
lm_over_time("lm",  
  <cpu_rate>[6h:1m],  
  (<compaction_rate> or <query_eval_rate>)[6h:1m],  
  "d",  
  "")
```

compaction $\approx 0.62 \times \text{busy } 3\% \approx 0.02 \text{ core}$
query_eval $\approx 0.34 \times \text{busy } 27\% \approx 0.09 \text{ core}$
intercept ≈ 0.60
 $r^2 \approx 0.75$

A weight is CPU while it runs; multiply by how often it runs.

query_eval (mostly rules): always on → 4–5× compaction's average cost, almost no spikes.

Compaction: rare bursts → it makes the spikes.

The parts add up: $0.60 + 0.02 + 0.09 \approx 0.71$ cores, the measured average CPU. Full query: appendix A6.



STATISTICAL CAVEATS & REALITY CHECKS

Where `lm_over_time` Breaks in Practice

Monitoring predictors are often autocorrelated, collinear, non-stationary and coarsely sampled. One-driver fits hold up; multi-driver coefficients are mostly part-artifact.

Collinear Variables

Two drivers that always move together: the model can't tell who did it, so it guesses.

This Prometheus: head GC always runs right after compaction. Add it as a suspect and compaction drops 0.62 $\rightarrow$ 0.43 while head_gc gets a weight of 14.3. Same CPU, credit split at random.

"Touch" it, and it flips

Same box, same query, a few days apart: query_eval went 1.54 $\rightarrow$ 0.30, go_gc +0.40 $\rightarrow$ -1.99.

If a small change in window flips a weight, that weight was never real.

Fine now, holes later

An instant query can fit cleanly while some steps of the range graph can't: a driver went flat, or copied another, so its weight is dropped (NaN).

One good "isolated" number proves nothing.



Uglier: `timeseries_gen()` creates synthetic data that never touches the TSDB.

Why it exists

Test queries, panels, or rules against specific shapes without ingesting fake data, or fabricating it in recording-rules.

Why it's f*dangerous

A query engine that can fabricate series can be abused (EHLO @jjo), screenshoted. The blast radius includes demos that look like real data, making it hard to defend in a default build.



Series that never touched the TSDB

<code>{env="prod"}</code>	1
<code>{env="stage"}</code>	1
<code>{env="canary"}</code>	1

just look at this ugliness:

```
timeseries_gen(  
  '{{rangeSeries "env" "prod,stage,canary" 1.0}}'  
)
```

which helps post-filtering my heavy promQLs as:

```
[ ...  
my 50+lines long promQL with joins and whatnots  
...]  
* on(env) group_left()  
timeseries_gen(...)
```

A Go template that **conjures series out of thin air**, once per query, up to 10,000 of them.

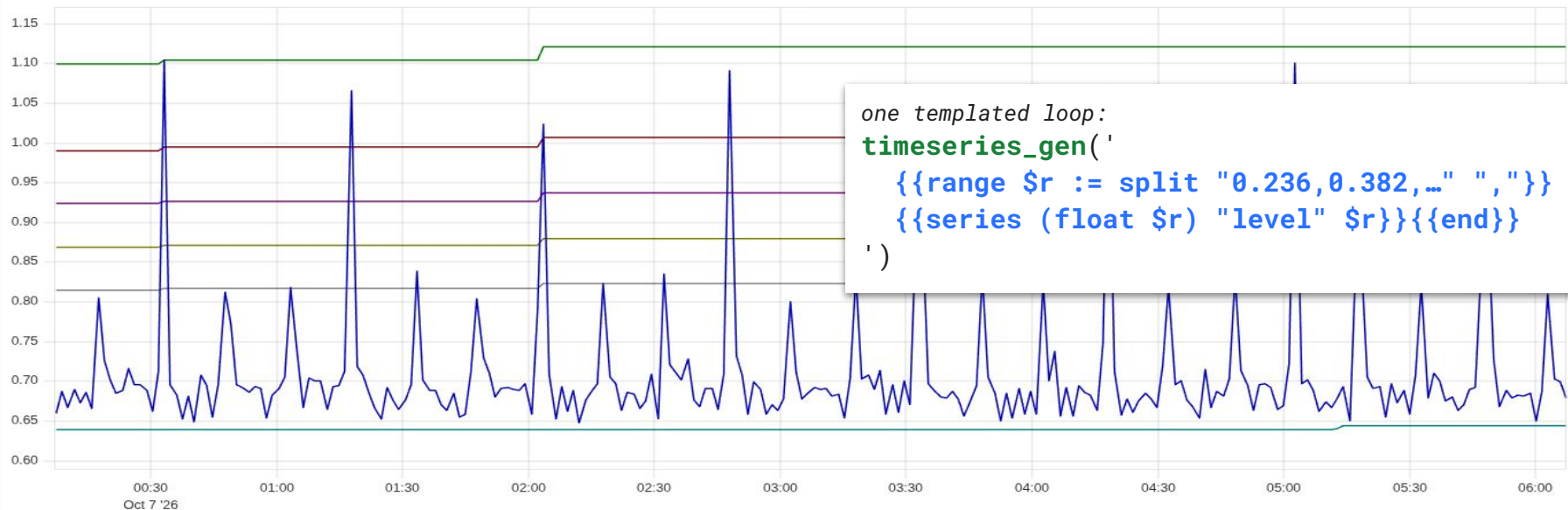
Handy for join filters and testing rules.

Also handy for fake screenshots and/or bragging to your friends.

Full query: appendix A7.



Fibonacci levels, for the traders in the room



- {level="0%"}
- {level="100%"}
- {level="23.6%"}
- {level="38.2%"}
- {level="50%"}
- {level="61.8%"}
- {level="cpu"}

Click: show single series, Ctrl + click: hide single series

Swing high = 0%, swing low = 100%, FIB ratios in between:
constants from `timeseries_gen`, stretched by
`max/min_over_time`.

Our CPU dutifully bounces off "support". Statistically
meaningless, but now you can draw it in PromQL ;)



What earns a place in the query engine? opinionated

SHIP (possibly;)

correlation_over_time
cheap, safe

regression_over_time
expected values

ewma_over_time
better trends

time_to_threshold
ergonomic rearranged math

MAYBE :?

zscore / robust_zscore
needs re²visiting :P

timeseries_gen
rename to `vector\_gen()`?
handy but dangerous :)

NO :(

lm_over_time
a **footgun** with "very" confident coefficients



FIN.



but let's keep those
clippers *sharp* ;)



- repo: <https://github.com/jjo/prometheus> branch: jjo/all-add-branches
- container images: <https://hub.docker.com/r/xjio/prometheus/tags?name=jjo-all>

Appendix

Full queries behind the live slides

Every query ran against one homelab Prometheus (um cluster, single replica) built from jjo/all-add-branches with `--enable-feature=promql-experimental-functions`.

Each query is shown exactly as executed, formatted for reading; each parses to the same expression as the one that produced its screenshot.

- A1 `ewma_over_time()` — range 3h
- A2 `zscore()` & `robust_zscore()` — range 6h
- A3 `time_to_threshold()` — instant
- A4 `correlation_over_time()` — range 12h
- A5 `regression_over_time()` — range 6h
- A6 `lm_over_time()` — instant
- A7 `timeseries_gen()` — instant
- A8–A10 “Can't it be done today?” — plain-PromQL rewrites
- A11 `timeseries_gen()` — Fibonacci levels, range 6h



Meet the ghost plateau

```
label_replace(
    sum(rate(process_cpu_seconds_total{job="prometheus-stack-kube-prom-prometheus"}[1m])),
    "s", "raw", "", "")
or
label_replace(
    avg_over_time((sum(rate(process_cpu_seconds_total{job="prometheus-stack-kube-prom-prometheus"}[1m])))[10m:30s]),
    "s", "avg_over_time 10m", "", "")
or
label_replace(
    ewma_over_time((sum(rate(process_cpu_seconds_total{job="prometheus-stack-kube-prom-prometheus"}[1m])))[10m:30s], 0.2),
    "s", "ewma_over_time 10m, 0.2", "", "")
```



zscore > 3 is a unicorn

```
label_replace(  
  max(zscore(  
    sum by (instance) (node_load1{instance=~"172\\.16\\.16\\. (6[4-9]|7[0-4]):9100"})  
  )),  
  "s", "max zscore (9 nodes)", "", "")  
or  
label_replace(  
  max(robust_zscore(  
    sum by (instance) (node_load1{instance=~"172\\.16\\.16\\. (6[4-9]|7[0-4]):9100"})  
  )),  
  "s", "max robust_zscore (9 nodes)", "", "")  
or  
label_replace(  
  vector(3),  
  "s", "threshold 3", "", "")
```



Who's running out of disk?

```
sort(  
    time_to_threshold(kubelet_volume_stats_available_bytes[6h], 0) / 86400 > 0  
)
```



Everyone suspects the queries

```
label_replace(
  correlation_over_time(
    (sum(rate(process_cpu_seconds_total{job="prometheus-stack-kube-prom-prometheus"}[2m])))[1h:1m],
    (sum(rate(prometheus_tsdb_compaction_duration_seconds_sum{job="prometheus-stack-kube-prom-prometheus"}[2m])))[1h:1m]
  ),
  "vs", "compaction", "", "")
or
label_replace(
  correlation_over_time(
    (sum(rate(process_cpu_seconds_total{job="prometheus-stack-kube-prom-prometheus"}[2m])))[1h:1m],
    (sum(rate(prometheus_engine_query_duration_seconds_sum{job="prometheus-stack-kube-prom-prometheus", slice="inner_eval"}[2m])))[1h:1m]
  ),
  "vs", "query eval", "", "")
```



What does a compaction cost?

```
label_replace(
  sum(rate(process_cpu_seconds_total{
    job="prometheus-stack-kube-prom-prometheus",
    container="prometheus"
  }[2m])),
  "series", "actual", "", "")
or
label_replace(
  regression_over_time(
    (sum(rate(process_cpu_seconds_total{
      job="prometheus-stack-kube-prom-prometheus",
      container="prometheus"
    }[2m])))[6h:1m],
    (sum(rate(prometheus_tsdb_compaction_duration_seconds_sum{
      job="prometheus-stack-kube-prom-prometheus",
      container="prometheus"
    }[2m])))[6h:1m],
    "prediction"
  ),
  "series", "prediction", "", "")
```



Two suspects, one culprit

```
lm_over_time(  
  "lm",  
  (sum(rate(process_cpu_seconds_total{  
    job="prometheus-stack-kube-prom-prometheus",  
    container="prometheus"  
  }[2m])))[6h:1m],  
  (  
    label_replace(  
      sum(rate(prometheus_tsdb_compaction_duration_seconds_sum{  
        job="prometheus-stack-kube-prom-prometheus",  
        container="prometheus"  
      }[2m])),  
      "d", "compaction", "", "")  
    or label_replace(  
      sum(rate(prometheus_engine_query_duration_seconds_sum{  
        job="prometheus-stack-kube-prom-prometheus",  
        container="prometheus",  
        slice="inner_eval"  
      }[2m])),  
      "d", "query_eval", "", "")  
    )[6h:1m],  
  "d",  
  ""  
)
```



Series that never touched the TSDB

```
timeseries_gen('{{rangeSeries "env" "prod,stage,canary" 1.0}}')
```



Appendix A8 · “Can't it be done today?” — `time_to_threshold()` & `zscores`

Yes: these are exact rewrites (verified)

```
# time_to_threshold(x[6h], T)          (fits the line twice and fetches the window twice)
(T - predict_linear(x[6h], 0)) / deriv(x[6h])

# zscore(x)                          (across series)
(x - on() group_left avg(x)) / on() group_left stddev(x)

# zscore_over_time(x[1h])
(x - avg_over_time(x[1h])) / stddev_over_time(x[1h])

# robust_zscore(x)                   (mind the () after group_left)
(x - on() group_left quantile(0.5, x))
  / on() group_left() (1.4826 * quantile(0.5, abs(x - on() group_left quantile(0.5, x))))

# robust_zscore_over_time(x[1h])     (needs the experimental mad_over_time, upstream since 2.49)
(last_over_time(x[1h]) - quantile_over_time(0.5, x[1h])) / (1.4826 * mad_over_time(x[1h]))
```



Appendix A9 · “Can't it be done today?” — correlation & regression

Pearson and the slope: yes (if you know the trick)

```
# correlation_over_time(a[30m:30s], b[30m:30s])    (Pearson only; Spearman/Kendall: no way)
# polarization identity: cov(a, b) = (var(a + b) - var(a - b)) / 4

(  stdvar_over_time((a + b)[30m:30s])
  - stdvar_over_time((a - b)[30m:30s]) )
/ ( 4 * stddev_over_time((a and b)[30m:30s]) * stddev_over_time((b and a)[30m:30s]) )

# regression_over_time(y[30m:30s], x[30m:30s], "slope")    (same identity, ÷ var(x))

(  stdvar_over_time((y + x)[30m:30s])
  - stdvar_over_time((y - x)[30m:30s]) )
/ ( 4 * stdvar_over_time((x and y)[30m:30s]) )

# do NOT write the textbook  $E[ab] - E[a] \cdot E[b]$ : on ~1e8-sized series it returned  $r = 3.747$  (sic)
```



Appendix A10 · “Can't it be done today?” — ewma_over_time()

Yes, really. Now please un-see it.

```
# ewma_over_time(g[10m:30s], 0.2), exact to 1e-9 (verified with promqltest)
# needs the experimental end();  $\lambda = -\ln(1 - \alpha) / \text{step} = -\ln(0.8) / 30\text{s}$ 

(1 - 0.8 ^ count_over_time(g[10m:30s]))
* sum_over_time((g * scalar(exp(vector((0.22314355131420976 / 30) * (time() - end()))))) [10m:30s])
/ sum_over_time((g * 0 + scalar(exp(vector((0.22314355131420976 / 30) * (time() - end()))))) [10m:30s])
+ 0.8 ^ count_over_time(g[10m:30s]) * first_over_time(g[10m:30s])

# the catch: weights are relative to one global end(), so far from it exp() underflows  $\rightarrow 0/0 \rightarrow \text{NaN}$ 
#  $\alpha = 0.2$  @ 30s step: safe up to ~26h of range query       $\alpha = 0.9$  @ 15s step: ~1.3h
# and  $\lambda$  is welded to the subquery step: change [10m:30s]  $\rightarrow$  recompute  $\lambda$  by hand
```



Fibonacci levels, for the traders in the room

```
label_replace(sum(rate(process_cpu_seconds_total{job="prometheus-stack-kube-prom-prometheus", container="prometheus"}[2m])),
"level", "cpu", "", "")
or

scalar(max_over_time((sum(rate(process_cpu_seconds_total{job="prometheus-stack-kube-prom-prometheus", container="prometheus"}[2m]))
[6h:1m]))
- timeseries_gen(`{{series 0 "level" "0%"}}{{series 0.236 "level" "23.6%"}}{{series 0.382 "level" "38.2%"}}{{series 0.5 "level"
"50%"}}{{series 0.618 "level" "61.8%"}}{{series 1 "level" "100%"}}`)
*
scalar(max_over_time((sum(rate(process_cpu_seconds_total{job="prometheus-stack-kube-prom-prometheus", container="prometheus"}[2m]))
[6h:1m])) -
min_over_time((sum(rate(process_cpu_seconds_total{job="prometheus-stack-kube-prom-prometheus", container="prometheus"}[2m])))[6h:1m]
))
```

